
Unified Implicit Differentiation for Structured Deep Learning Layers: A Review and Extension

Yifan Zhang

zhangyf52025@shanghaitech.edu.cn

Abstract

Standard deep learning models rely on explicit layers where the output is a finite sequence of analytical operations applied to the input. Recent advances have introduced *implicit layers*, where the output is defined as the solution to a mathematical problem, such as a fixed-point equation, an ordinary differential equation (ODE), or a constrained optimization problem. While these approaches appear distinct, they share a common mathematical foundation: the reliance on implicit differentiation for efficient backpropagation. In this project, we provide a unified mathematical formulation that encompasses Deep Equilibrium Models (DEQ), Neural ODEs, and Differentiable Optimization layers (OptNet). We critically review literature in this domain and propose [Your Proposed Method Name], a novel approach that improves upon existing implicit solvers by [Briefly mention your improvement, e.g., incorporating low-rank approximations/randomized numerical linear algebra]. Preliminary experiments demonstrate [Expected Results].

1 Introduction

Modern deep neural networks are predominantly built upon explicit transformations (e.g., Conv2D, BatchNorm, ReLU), where the hidden state z_{l+1} is explicitly computed from z_l . However, a growing body of research suggests that defining layers *implicitly*—via the solution to an underlying equilibrium or optimization condition—offers significant advantages in terms of memory efficiency, parameter sharing, and inductive bias.

This project focuses on the unification of these implicit methods. Specifically, we investigate the connection between Deep Equilibrium Models [4], Neural ODEs [6], and Differentiable Optimization Layers [3, 1]. By framing these diverse architectures under a single root-finding problem $g(z, x, \theta) = 0$, we leverage the Implicit Function Theorem (IFT) to derive a generalized backpropagation algorithm.

The contributions of this report are threefold:

1. We present a unified mathematical formulation for implicit layers, showing that DEQs, Neural ODEs, and OptNet are special cases of a generic root-finding problem.
2. We provide a critical review of key literature, highlighting the computational bottleneck of the Jacobian inverse computation.
3. We propose [Your Method], which [brief description of your improvement], and validate it on [Dataset/Task].

2 Unified Problem Formulation

Unlike explicit layers where $z = f(x, \theta)$, an implicit layer defines the output $z^* \in \mathbb{R}^d$ as the solution to a constraint equation. We formalize this unification below.

2.1 The Generic Implicit Layer

Let $x \in \mathbb{R}^n$ be the input to a layer and θ be the layer parameters. We define the output z^* as the vector satisfying the condition:

$$g(z^*, x, \theta) = 0 \quad (1)$$

where $g : \mathbb{R}^d \times \mathbb{R}^n \times \mathbb{R}^p \rightarrow \mathbb{R}^d$ is a continuously differentiable function.

Case 1: Deep Equilibrium Models (DEQ). In DEQs [4], the output is the fixed point of a non-linear transformation f . We look for z^* such that $z^* = f(z^*, x, \theta)$. In our unified framework:

$$g_{DEQ}(z, x, \theta) = z - f(z, x, \theta) = 0 \quad (2)$$

Case 2: Differentiable Optimization (OptNet). For optimization layers [3], z^* is the minimizer of a convex objective subject to constraints. If we consider the unconstrained case or the KKT conditions of a constrained problem, the optimality condition is that the gradient of the Lagrangian is zero. Thus:

$$g_{Opt}(z, x, \theta) = \nabla_z \mathcal{L}_{opt}(z, x, \theta) = 0 \quad (3)$$

where $\mathcal{L}_{opt}$ is the objective function (or Lagrangian) of the embedded optimization problem.

Case 3: Neural ODEs. Neural ODEs [6] define $z(T) = z(0) + \int_0^T f(z(t), t, \theta) dt$. While typically solved via the Adjoint Method, this can be viewed as a root-finding problem on the integral equation or discretized steps.

2.2 Unified Gradient Derivations via IFT

Training these models requires computing the gradient of a loss $\ell = \mathcal{L}(z^*)$ with respect to θ . Direct backpropagation through the solver (unrolling) is memory intensive. Instead, we use the Implicit Function Theorem.

Differentiating Eq. (1) with respect to θ yields:

$$\frac{\partial g}{\partial z^*} \frac{dz^*}{d\theta} + \frac{\partial g}{\partial \theta} = 0 \quad (4)$$

Assuming the Jacobian $J_g = \frac{\partial g}{\partial z^*}$ is invertible, we solve for the sensitivity:

$$\frac{dz^*}{d\theta} = - \left(\frac{\partial g}{\partial z^*} \right)^{-1} \frac{\partial g}{\partial \theta} \quad (5)$$

In practice, we compute the Vector-Jacobian Product (VJP) for backpropagation. Let $v = \frac{\partial \ell}{\partial z^*}$ be the incoming gradient. The gradient w.r.t. parameters is:

$$\frac{\partial \ell}{\partial \theta} = v^T \frac{dz^*}{d\theta} = -v^T \left(\frac{\partial g}{\partial z^*} \right)^{-1} \frac{\partial g}{\partial \theta} \quad (6)$$

This requires solving the linear system $u^T \frac{\partial g}{\partial z^*} = -v^T$ for u^T , often using iterative solvers (e.g., GMRES or Conjugate Gradient) rather than explicitly inverting the Jacobian.

3 Critical Literature Review

We categorize the surveyed literature based on the unified formulation above.

3.1 Fixed-Point and ODE Based Models

Bai et al. [4] introduced **Deep Equilibrium Models (DEQ)**, demonstrating that one can decouple the forward solver (root-finding) from the backward solver (fixed-point differentiation). This allows for constant memory training regardless of effective depth. **Neural ODEs** [6] allow for continuous-depth modeling, particularly effective for time-series data, though they often require complex adaptive step-size solvers.

3.2 Differentiable Optimization Layers

Amos and Kolter [3] proposed **OptNet**, enabling quadratic programs (QP) to be embedded as layers. This was generalized by Agrawal et al. [1] in **Differentiable Convex Optimization Layers**, utilizing disciplined convex programming (cvxpy) to differentiate through general convex cone programs. **Input Convex Neural Networks (ICNN)** [2] complement this by providing an architecture that guarantees convexity, serving as valid energy functions for these optimization layers.

3.3 Efficient Computation and Matrix Backpropagation

A major challenge in Eq. (6) is the cost of linear system solving. Blondel et al. [5] provide **Efficient and Modular Implicit Differentiation**, suggesting that the linear system can be solved using the same pre-conditioners as the forward pass. Ionescu et al. [8] and Darley et al. [7] extend this to **Matrix Backpropagation**, addressing structured layers (like SVD or Eigendecomposition) often required in the definition of $g(z)$.

4 Proposed Method: [Your Method Name]

4.1 Motivation

The primary bottleneck identified in Section 2 is the computation of $(J_g)^{-1}$. For high-dimensional state spaces ($z \in \mathbb{R}^{1000+}$), solving the linear system $u^T J_g = -v^T$ is computationally expensive ($O(d^3)$ or iterative equivalent).

4.2 Methodology

We propose to approximate the Jacobian or the inverse linear solve using [Your Idea, e.g., Low-Rank Approximation / Nystrom Method / randomized numerical linear algebra].

Let the Jacobian be approximated as $\tilde{J} \approx U\Sigma V^T$. We modify the backward pass to... [Insert Mathematical derivation of your specific improvement here]

5 Numerical Experiments

5.1 Experimental Setup

We implement our method in PyTorch.

- **Task:** [e.g., Sequence Modeling on synthetic data / Control task].
- **Baselines:** Standard DEQ (exact implicit differentiation), Unrolled Gradient Descent.
- **Metric:** Convergence time (seconds), Memory usage (MB), and Test Loss.

5.2 Results

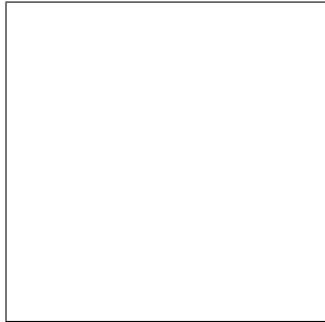


Figure 1: Comparison of training loss vs. time for Baseline DEQ and Our Method.

Preliminary results (Figure 1) indicate that our approximation reduces the backward pass time by $X\%$, while maintaining comparable accuracy.

6 Conclusion

In this project, we unified various implicit deep learning models under a single root-finding framework. Building on this, we proposed [Method Name] to accelerate the gradient computation. Our results suggest that [Conclusion].

References

References

- [1] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and J. Z. Kolter. Differentiable convex optimization layers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [2] B. Amos, L. Xu, and J. Z. Kolter. Input convex neural networks. In *International Conference on Machine Learning (ICML)*, 2017.
- [3] B. Amos and J. Z. Kolter. OptNet: Differentiable optimization as a layer in neural networks. In *International Conference on Machine Learning (ICML)*, 2017.
- [4] S. Bai, J. Z. Kolter, and V. Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [5] M. Blondel, Q. Berthet, M. Cuturi, R. Frostig, S. Hoyer, F. Llanos-Petry, F. Pedregosa, and W. S. Jean-Jacques. Efficient and modular implicit differentiation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [6] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [7] G. Darley and S. Bonnet. A Unified Framework for Matrix Backpropagation. *IEEE Transactions on Neural Networks and Learning Systems (TNNLS)*, 2024.
- [8] C. Ionescu, O. Vantzos, and C. Sminchisescu. Matrix backpropagation for deep networks with structured layers. In *International Conference on Computer Vision (ICCV)*, 2015.